

DOCUMENTACIÓN TÉCNICA

Sistema Integral de Gestión Financiera, Contable y Empresarial

Denominación de trabajo: Cuadra

Cuadra constituye una denominación de trabajo utilizada durante el desarrollo y no implica registro marcario.

Autora: Julieta Arrazate
Versión documentada: v3.12 - Junio 2026

1. ARQUITECTURA GENERAL

El sistema implementa una arquitectura de tres capas (three-tier) con separación clara de responsabilidades:

La arquitectura se divide en tres capas principales:

- Capa de Presentación (Frontend):** Interfaz de usuario que permite la interacción con el sistema. Se desarrolla utilizando tecnologías como React Native y Web App (PWA).
- Capa de Negocio (Backend):** Lógica de negocio que maneja los datos y las operaciones. Se desarrolla utilizando tecnologías como Django REST Framework y SQLAlchemy ORM.
- Capa de Datos (Database):** Almacén de datos que almacena la información del sistema. Se utiliza PostgreSQL como base de datos.

Además, el sistema incluye componentes de seguridad y autenticación, como JWT y OAuth2, para garantizar la integridad de los datos y la privacidad de la información.

2. ARQUITECTURA DEL BACKEND (FastAPI)

2.1 Estructura de capas internas

La estructura de capas internas del backend se define de la siguiente manera:

- app/**: Directorio principal que contiene los módulos de la aplicación.
- main.py**: Punto de entrada de la aplicación.
- config.py**: Configuración de la aplicación.
- database.py**: Configuración de la base de datos.
- middleware/auth.py**: Middleware de autenticación y permisos.
- models/**: Modelos de datos.
- routers/**: Endpoints HTTP.
- services/**: Lógica de negocio.
- schemas/**: Esquemas de validación.

2.2 Ciclo de vida de la aplicación (lifespan)

Al iniciar, el servidor ejecuta en orden:

- Migraciones automáticas:** Se ejecuta `run_alembic()` para aplicar migraciones pendientes.
- Safety nets:** Se crean tablas de seguridad para columnas nuevas que no existían en las instancias legadas.
- Plan de cuentas:** Se crean cuentas contables nuevas sin sobrescribir las existentes (idempotente).
- Backfill:** Se vinculan clientes existentes a sus cuentas contables por nombre normalizado.
- Cleanup de tokens vencidos:** Se limpian los tokens de acceso expirados.
- Scheduler APScheduler:** Se inician tareas programadas (backup, alertas, etc.).

2.3 Encodificación de tipos numéricos

Se implementa un encoder JSON personalizado (`DecimalEncoder`) para serializar transparentemente los tipos `Decimal` (devueltos por SQLAlchemy para columnas `Numeric`) como valores de punto flotante en las respuestas JSON, sin requerir conversiones explícitas en cada router.

2.4 Organización de routers

Los routers se registran con prefijo `/` y etiquetas de OpenAPI. Los endpoints públicos (sin autenticación) se montan en `/p/` y utilizan validación por token de 7 días en lugar de JWT.

3. ARQUITECTURA DEL FRONTEND (React)

3.1 Carga diferida de rutas

Todas las páginas se cargan con `React.lazy()` mediante la función auxiliar `lazyPage<M, K>()`, que proporciona tipado estático completo sobre el módulo y la exportación. Solo `Login`, `RecuperarPassword` y `RestablecerPassword` se cargan de forma ansiosa para garantizar el primer render inmediato.

3.2 Gestión de estado global

Seis stores independientes con Zustand:

Store	Responsabilidad
auth	Usuario autenticado, token JWT, permisos, roles
org	Organización activa, cambio de org
theme	Tema light/dark, persistencia en localStorage
lock	Estado de bloqueo PIN/biometría
confirm	Diálogo de confirmación global
toast	Notificaciones de usuario

3.3 Cliente HTTP centralizado (api.ts)

Un único módulo (~25 KB) encapsula todas las llamadas a la API con:

- `Interceptor` de request: inyección automática del token JWT y `org_id` activo
- `Interceptor` de respuesta: normalización de errores 422 de Pydantic a string legible
- `Retry` automático: 3 reintentos con backoff lineal (1.5s/3s/4.5s) ante 502/503/504 y errores de red en GET
- `Keep-alive`: ping a `/health` cada 14 minutos para mantener el backend activo

3.4 Progressive Web App (PWA)

- Service Worker con estrategia `network-first` y fallback a caché

- Web App Manifest para instalaci3n en dispositivos m3viles
- Share Target API para recibir archivos desde otras aplicaciones
- Web Push API para notificaciones nativas
- AppLockGuard con PIN + WebAuthn (biometr3a) en dispositivos compatibles

3.5 Soporte de tema (dark/light mode)

Script s3ncrono en <head> de index.html para evitar el flash de tema incorrecto antes de que React hidrate. El tema se persiste en localStorage y se aplica mediante clases TailwindCSS.

4. BASE DE DATOS

4.1 Motor y configuraci3n

- **Motor:** PostgreSQL (versi3n cloud managed)
- **ORM:** SQLAlchemy 2.0 con sesiones s3ncronas y pool de conexiones
- **Migraciones:** Alembic con historial de 9 versiones
- **Aritm3tica financiera:** todas las columnas monetarias usan Numeric(12, 2) para precisi3n exacta sin errores de punto flotante

4.2 Modelos principales

Modelo	Tabla	Descripci3n
Organizacion	organizaciones	Entidad multi-tenant ra3z
User	users	Usuarios del sistema con roles y permisos
Cliente	clientes	Directorio de entidades pagadoras
ExtractoBancario	extractos_bancarios	Cabecera de extracto
MovimientoBanco	movimientos_banco	L3neas del extracto bancario
Planilla	planillas	Cabecera de planilla de pagos
PlanillaRow	planilla_rows	Filas de planilla con estado de conciliaci3n
Cheque	cheques	Cheques con ciclo de vida completo
Portador	portadores	Cat3logo de portadores de cheques
ArqueoDiario	arqueos_diarios	Arqueos de caja con denominaciones
Egreso	egresos	Pagos y gastos unificados
CategoriaEgreso	categorias_egreso	Categor3as de egresos configurables
Liquidacion	liquidaciones	Liquidaciones peri3dicas
LiquidacionDetalle	liquidacion_detalle	L3neas de liquidaci3n
PlanCuenta	plan_cuentas	Plan de cuentas jer3rquico
Asiento	asientos	Asientos contables
AsientoDetalle	asiento_detalle	L3neas de asiento (partida doble)
AuditoriaLog	auditoria_logs	Log inmutable de operaciones
PatronAprendido	patrones_aprendidos	Patrones de conciliaci3n autom3tica

4.3 Columnas de control de soft delete

Los modelos con soft delete incluyen: - deleted_at (timestamp): fecha de eliminaci3n l3gica - deleted_by_id (FK): usuario que elimin3 el registro - Queries con filtro WHERE deleted_at IS NULL para excluir registros eliminados

4.4 Migraciones de esquema

Migraci3n	Descripci3n
001_baseline	Esquema inicial con todos los modelos fundacionales
002_soft_delete	Columnas de eliminaci3n l3gica
003_password_reset	Modelo PasswordResetToken
004_performance_indexes	3ndices en columnas de b3squeda frecuente
005_revoked_tokens	Tabla de tokens JWT revocados
006_unique_constraints	Restricciones de unicidad en datos cr3ticos
007_float_to_numeric	Conversi3n de columnas Float a Numeric(12,2)
008_cliente_comision	Campo porcentaje_comision en clientes
009_drop_tablas_viejas	Eliminaci3n de tablas de m3dulos deprecados

5. SEGURIDAD

5.1 Autenticaci3n

Mecanismo	Detalle
JWT	Tokens firmados con clave secreta, expiraci3n 8h (4h para rol contador)
2FA por email	C3digo de 6 d3gitos con hash SHA-256, TTL 10 minutos (admin y superadmin)
PIN de bloqueo	C3digo num3rico de 6 d3gitos, verificaci3n local en el dispositivo
WebAuthn (biometr3a)	Autenticaci3n con huella o Face ID en dispositivos compatibles
Tokens revocados	Tabla RevokedToken para invalidaci3n expl3cita de JWT

5.2 Autorizaci3n (control de acceso)

El sistema implementa un modelo de permisos en 3 capas:

Capa 1 3 roles: - SUPERADMIN: acceso completo a todas las organizaciones - ADMIN: gesti3n completa de su organizaci3n - OPERADOR: operatoria diaria sin borrado de datos - REVISOR: solo lectura contable - AUDITOR: lectura contable y financiera - CONTADOR: operatoria + contabilidad en solo lectura + login por aprobaci3n

Capa 2 3 permisos funcionales: - upload_files: subir extractos y planillas - reconcile: conciliar y operar planillas - manage_finance: caja, cheques, pagos, cuentas corrientes - view_accounting: libro diario, mayor, sumas y saldo - admin_accounting: plan de cuentas, reglas, backfill, reset - manage_users: usuarios, organizaciones, liquidaciones - view_audit: m3dulo de auditor3a - delete_records: operaciones de borrado (solo Admin y Superadmin)

Capa 3 3 Guard en backend:

```
require_permission("nombre_permiso") # Decorador en dependencias FastAPI
```

5.3 Seguridad de red y endpoints

Medida	Implementaci3n
Rate limiting	slowapi 3 l3mites configurados en endpoints de auth
CORS	M3todos y headers expl3citos, sin wildcard
Headers HTTP de seguridad	X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, Referrer-Policy, Permissions-Policy
HTTPS forzado	Configurado en plataforma de despliegue
Validaci3n de entrada	Pydantic v2 en todos los endpoints con tipos estrictos

5.4 Flujo de login especial para rol Contador

1. El contador env   credenciales     servidor no devuelve token
2. Se crea un LoginApproval en estado pending (expira en 10 minutos)
3. Los superadmins reciben notificaci    push
4. El cliente del contador hace polling a /auth/login-approval/{id}
5. Al aprobar, el servidor genera JWT con expiraci   n de 4 horas
6. Token entregado una    nica vez (invalidado tras el primer uso)

6. APIs Y ENDPOINTS PRINCIPALES

6.1 Grupos de endpoints

Prefijo	MÃ³dulo	AutenticaciÃ³n
/auth/*	AutenticaciÃ³n y 2FA	PÃºblica + protegida
/me	Perfil del usuario	JWT
/admin/*	GestiÃ³n de usuarios	JWT + manage_users
/extractos/*	Extractos bancarios	JWT + upload_files
/planillas/*	Planillas de pagos	JWT + upload_files/reconcile
/clientes/*	Directorio de clientes	JWT
/cheques/*	GestiÃ³n de cheques	JWT + manage_finance
/pagos/*	MÃ³dulo de egresos	JWT + manage_finance
/caja/*	Caja y arqueos	JWT + manage_finance
/contabilidad/*	Contabilidad	JWT + view/admin_accounting
/liquidaciones/*	Liquidaciones	JWT + reconcile/manage_users
/auditoria/*	Log de auditoriÃ¡a	JWT + view_audit
/analisis/*	Reportes y alertas	JWT
/search/*	BÃ¡squeda global	JWT
/agente/*	IA y OCR	JWT
/p/*	Acceso pÃºblico	Token de 7 dÃas (sin JWT)

6.2 Formato de respuestas

Todas las respuestas utilizan JSON. Los tipos `decimal` de SQLAlchemy se serializan automáticamente como números de punto flotante mediante el encoder personalizado `DecimalEncoder`. Los errores siguen el formato FastAPI estáñdar con campo `detail`.

7. INTEGRACIONES EXTERNAS

7.1 Google Gemini (IA y OCR)

Configuraci3n: variable de entorno GEMINI_API_KEY. Si no est3 presente, las funciones de IA y OCR quedan desactivadas sin errores.

Modelo utilizado: gemini-2.5-flash (configurable via GEMINI_MODEL)

Uso: - **Chat IA:** function calling con 5 funciones que acceden a datos reales de la base de datos - **OCR cheques:** env a la imagen como inline_data y extrae n mero, banco, librador, monto, fechas - **OCR comprobantes:** extrae monto, fecha, beneficiario y referencia de comprobantes de transferencia

Manejo de errores: clasificaci3n de errores de API (clave inv3lida, cuota agotada, modelo no disponible) con mensajes amigables. Reintento autom3tico con espera de 5 segundos ante errores de l3mite por minuto.

7.2 Resend (Email)

Configuraci3n: variable de entorno RESEND_API_KEY. Sin ella, las funciones de email se deshabilitan (degradaci3n elegante).

Uso: - CÃ3digo 2FA (asunto: â€œCÃ3digo de verificaciÃ3nâ€) - RecuperaciÃ3n de contraseÃ±a (link de restablecimiento) - Backup diario del sistema (adjunto JSON gzipeado)

7.3 Web Push / VAPID

Configuraci3n: variables VAPID PUBLIC KEY y VAPID PRIVATE KEY. Sin ellas, el scheduler de alertas no se activa.

Uso: - Alertas de cheques próximos a vencer (3 días) - Movimientos sin conciliar por más de 7 días - Notificación de solicitud de sesión de rol contador - Push de prueba desde el perfil de administrador

7.4 AWS S3 / Cloudflare R2

Configuraci3n: variables S3_ENDPOINT, S3_BUCKET, S3_ACCESS_KEY, S3_SECRET_KEY, S3_PUBLIC_URL. Sin ellas, las fotos se almacenan como base64 en la base de datos.

Uso: almacenamiento de fotos de comprobantes de egresos y cheques.

7.5 Sentry

Configuraci3n: variables SENTRY_DSN (backend) y VITE_SENTRY_DSN (frontend). Opt-in, sin overhead si no se configuran.

Uso: captura de excepciones no manejadas en backend y frontend para monitoreo en producción.

8. AUDITORÃA

El módulo de auditoría registra automáticamente:

Campo	Descripción
usuario_id / usuario_nombre	Quién realizó la operación
accion	Tipo de operación (crear, editar, eliminar, conciliar, etc.)
entidad	Módulo afectado (planilla, cheque, egreso, etc.)
entidad_id	ID del registro afectado
datos_antes	Estado del registro antes del cambio (JSON)
datos_despues	Estado del registro después del cambio (JSON)
ip_address	Dirección IP del cliente
created_at	Timestamp UTC del evento
organizacion_id	Organización en contexto

Los datos JSON del log son inmutables una vez escritos. La serializaci3n de tipos Decimal se resuelve con conversi3n recursiva antes de persistir.

9. SCHEDULER (TAREAS PROGRAMADAS)

Implementado con APScheduler en proceso, se inicia en el lifespan de FastAPI:

Tarea	Hora (ART)	Condición de activación
Backup JSON gzipeado por email	03:00	RESEND_API_KEY seteada
Alertas push (cheques + movimientos)	10:00	VAPID_PRIVATE_KEY seteada
Cleanup de tokens vencidos	Al iniciar	Siempre

10. TIMEZONE Y FECHAS

El servidor corre en UTC. Todas las fechas de negocio (transacciones, operaciones, arqueos, cheques, asientos) se calculan en UTC-3 (America/Argentina/Buenos_Aires) mediante el servicio `app/services/tz.py`:

```
def hoy_art() -> date:      # date.today() en timezone ART
def now_art() -> datetime:  # datetime.now() en timezone ART
```

Las marcas de auditoría (`created_at`) y expiración de tokens de seguridad permanecen en UTC, lo cual es correcto por convención.

El frontend usa el helper `src/utils/fecha.ts` para construir fechas desde los componentes locales del calendario (`año±o/mes/día`) en lugar de `toISOString()` que devuelve UTC.

11. PROCESAMIENTO OCR

11.1 Flujo de OCR de cheques

- Usuario adjunta fotografía desde el formulario
- El frontend comprime la imagen (canvas JPEG con fondo blanco)
- Se envía como base64 a `POST /agente/ocr-cheque`
- El backend envía a Gemini Flash con prompt específico
- Gemini devuelve JSON con: número, banco, librador, monto, fecha de emisión, fecha de depósito
- El frontend pre-completa solo los campos vacíos del formulario

11.2 Flujo de OCR de comprobantes de transferencia

- Usuario adjunta foto del comprobante en el módulo de egresos
- El frontend comprime la imagen
- Se envía a `POST /agente/ocr-transferencia`
- El backend extrae: monto, fecha, beneficiario, referencia
- El frontend pre-completa campos vacíos

11.3 Parseo de montos en formato argentino

Se implementa el helper `parseMonto()` para convertir formatos numéricos argentinos ("15.000,50"), estadounidenses ("15,000.50") y planos ("15000.5") a tipo numérico estándar antes de guardar en el formulario.

12. MOTOR DE CONCILIACIÓN

Ver documento `ACTIVOS_PI.md` para descripción detallada del algoritmo.

Entrada: extracto bancario (filas de `MovimientoBanco`) + planilla de pagos (filas de `PlanillaRow`)

Proceso: 1. Normalización de texto (NFKD unicode, mayúsculas, tokens) 2. Extracción de identificadores por tipo (CUIT, CBU, número de cuenta, referencia, titular) 3. Scoring multi-criterio para cada par (movimiento, fila) 4. Selección del movimiento con mayor score para cada fila 5. Actualización de estado y registro de auditoría

Salida: planilla con estado por fila (ok, pendiente, revisar) y movimiento asignado